PoolGrad: A Memory-Aware Runtime for understanding Compute–Memory Tradeoffs

Priyam Ghosh

Abstract

The performance of neural network training is governed by the interaction between computation, memory behavior, and execution strategy. In modern systems, these factors are tightly coupled, making it difficult to isolate and understand their individual contributions.

We present *PoolGrad*, a minimal, memory-aware runtime designed to study these interactions in a controlled setting. PoolGrad integrates a reverse-mode automatic differentiation engine, multiple matrix multiplication kernels (naive, tiled, and packed with SIMD), a Strassen-style algorithmic variant, a shape-aware scheduler, and a gradient memory system with planner-driven reuse.

Our experiments show that hardware-aligned optimizations such as data packing and SIMD vectorization dominate performance, achieving up to 2.7× speedup over naive implementations. In contrast, algorithmic approaches that reduce multiplication count introduce additional memory traffic and overhead, often resulting in inferior runtime performance. Memory optimizations significantly reduce allocation overhead (up to 90%) but have limited impact on overall execution time.

These results highlight a key systems insight: reducing arithmetic complexity alone is insufficient for performance gains unless the computation aligns with hardware execution characteristics, particularly memory access patterns and vectorization.

1 Introduction

Matrix multiplication (GEMM) is the dominant computational primitive in neural network training, accounting for a significant portion of both forward and backward pass execution. As a result, optimizing GEMM has been a central focus of machine learning systems and high-performance computing.

Two broad approaches have emerged for improving GEMM performance. The first focuses on *hardware-aware optimization*, including techniques such as tiling, data packing, and SIMD vectorization, which aim to maximize cache locality and utilize processor capabilities efficiently. The second focuses on *algorithmic optimization*, where the number of arithmetic operations is reduced through mathematical transformations, such as Strassen-style decompositions.

While both approaches target the same operation, their practical impact differs significantly. Modern CPUs are often limited not by raw arithmetic throughput but by memory bandwidth, cache behavior, and data movement. Consequently, reducing the number of multiplications does not necessarily translate into faster execution. Instead, performance is frequently dominated by how well computation aligns with the underlying hardware execution model.

In addition to computation, memory behavior plays a critical role in overall performance. Temporary buffers, gradient storage, and allocation overhead can influence both runtime and system stability. However, in many existing frameworks, compute and memory optimizations are tightly integrated, making it difficult to study their individual effects.

To address this, we introduce *PoolGrad*, a minimal runtime designed for controlled experimentation with compute–memory tradeoffs. PoolGrad implements a reverse-mode automatic differentiation engine, a unified kernel interface supporting multiple GEMM implementations, a shape-aware scheduler, and a memory system with planner-driven gradient reuse. The system is intentionally simplified to expose the interaction between computation, memory allocation, and execution strategy without the complexity of full-scale frameworks.

Using this setup, we evaluate three classes of optimizations: hardware-aware compute optimizations (tiling, packing, SIMD), memory reuse through pooling and planning, and algorithmic reduction of multiplications via a Strassen-style variant. Our results demonstrate that hardware-aligned optimizations consistently dominate runtime performance, while memory optimizations primarily affect allocation behavior. Algorithmic approaches, despite reducing arithmetic complexity, often underperform due to increased memory traffic and overhead.

The goal of this work is not to introduce a new high-performance kernel, but to provide a clear and reproducible framework for understanding the interaction between computation and memory in modern machine learning workloads.

2 System Design

PoolGrad is designed as a minimal runtime to expose the interaction between computation, memory management, and execution strategy. The system separates these concerns explicitly, allowing each component to be analyzed in isolation while still interacting during execution.

The runtime operates in two phases: a forward pass that constructs a computation graph, and a backward pass that executes gradient propagation. Both phases share a common kernel layer and memory system, ensuring that compute and memory behaviors are consistent across execution.

2.1 Execution Model

Execution proceeds in two stages:

Forward pass: Each operation produces a tensor and appends a node to a dynamic computation graph. Tensors are stored in a centralized `TensorStore`, and nodes record their input dependencies along with the operation type.

Backward pass: Gradients are computed by traversing the graph in reverse topological order. Each node applies its corresponding backward rule and accumulates gradients into its inputs.

Code available at: https://github.com/PriGoistic/poolgrad.

Unlike static graph frameworks, PoolGrad does not perform ahead-of-time compilation or graph optimization. Instead, execution is driven directly by the graph structure, making all kernel invocations and memory operations explicit.

2.2 Tensor Store and Graph Representation

All tensors are stored in a contiguous `TensorStore`, where each tensor is referenced by an integer identifier. This design avoids pointer-heavy structures and enables efficient indexed access during both forward and backward passes.

Each graph node contains:

- input tensor identifiers
- the operation type
- a backward function encoding gradient propagation

This representation enables efficient reverse traversal, explicit dependency tracking, and integration with memory planning.

2.3 Kernel Layer

All matrix multiplication operations are routed through a unified kernel interface. This includes both forward computation and backward gradient computation.

Backward matmul is implemented using implicit transpose variants:

$$\frac{\partial L}{\partial A} = \frac{\partial L}{\partial C} \cdot B^T, \quad \frac{\partial L}{\partial B} = A^T \cdot \frac{\partial L}{\partial C}$$

These operations are executed without materializing transposed matrices, reducing memory overhead and improving cache locality.

The kernel layer supports multiple implementations:

- **Naive:** A baseline implementation using straightforward triple loops.
- **Tiled:** Improves cache locality by operating on sub-blocks of the input matrices.
- **Packed + SIMD:** Reorganizes matrix data into contiguous panels and leverages vector instructions (e.g., NEON or AVX2) to increase arithmetic throughput.
- **MP Variant:** A Strassen-style decomposition that reduces multiplication count at the cost of additional additions and temporary buffers.

Packed kernels improve performance by reducing cache misses and enabling efficient vectorization, while SIMD instructions increase the number of operations performed per cycle.

2.4 Scheduler

Kernel selection is handled by a lightweight scheduler that maps matrix shapes (m, n, p) to a kernel strategy. The scheduler uses heuristic thresholds to balance overhead and performance:

- small matrices: naive kernel (low overhead)
- medium matrices: tiled kernel
- large matrices: packed + SIMD kernel
- MP variant: selectively enabled under size constraints

These decisions are based on empirical observations rather than analytical models. As a result, the scheduler is not globally optimal and may occasionally select suboptimal kernels for specific input sizes. However, this behavior is intentional, as it exposes the variability of kernel performance across different regimes.

2.5 Memory System and Lifetime Planning

PoolGrad employs a memory system focused on gradient buffer reuse. Instead of allocating new buffers for each operation, memory is reused based on tensor lifetimes.

The system consists of two components:

Memory pool: Maintains reusable buffers grouped by size. This reduces allocation overhead by recycling previously used buffers.

Lifetime planner: Performs backward liveness analysis over the computation graph. For each tensor, it determines the last step in the backward pass where its gradient is required. Buffers are then reused across tensors with non-overlapping lifetimes.

This approach enables deterministic reuse of memory, reducing allocation frequency while maintaining correctness. Unlike naive pooling, which relies on dynamic allocation and release, the planner provides structured reuse based on execution order.

Memory usage is tracked across multiple categories:

- *Gradient memory (live):* active gradient buffers
- *Gradient memory (reserved):* allocated buffer capacity
- *Activation memory:* tensor storage
- *Temporary memory:* intermediate buffers used by kernels

This separation allows precise analysis of how different components contribute to overall memory behavior.

2.6 Interaction Between Components

The runtime is structured so that each component influences execution in a distinct way:

- the graph determines execution order
- the scheduler determines kernel selection
- the kernel determines compute efficiency
- the memory system determines allocation behavior

These components interact during execution. For example, kernel selection affects temporary buffer usage, while memory reuse affects allocation overhead but not arithmetic cost. As a result, compute and memory optimizations are not independent, but their effects differ in magnitude and impact.

This explicit separation enables controlled experimentation and provides insight into the relative importance of different optimization strategies.

3 Experiments and Results

3.1 Experimental Setup

All experiments are conducted on a CPU-only environment (Apple M4, 10 cores) using Rust in release mode. Matrix multiplication workloads are evaluated on square matrices with sizes ranging from 32 to 512.

Each experiment is executed with fixed random seeds and includes warmup iterations to stabilize performance. Reported runtimes correspond to median values over multiple trials.

We evaluate three aspects of the system:

- kernel performance (compute efficiency)
- memory allocation behavior (pooling and planning)
- interaction between kernel execution and memory reuse

3.2 Kernel Performance

We first isolate compute behavior by comparing kernel implementations.

Size	Kernel	Time (ms)	Speedup
64	Naive	0.208	1.00
64	Packed+SIMD	0.023	9.04
128	Naive	0.310	1.00
128	Packed+SIMD	0.156	1.99
256	Naive	2.115	1.00
256	Packed+SIMD	0.902	2.34
512	Naive	15.198	1.00
512	Packed+SIMD	5.649	2.69

Table 1: Kernel performance comparison.

Packed + SIMD kernels consistently outperform other implementations at moderate and large sizes. The performance advantage increases with problem size, indicating improved utilization of vector units and more efficient cache behavior.

In contrast, tiled kernels provide only moderate improvements over the naive baseline. While tiling improves locality, it does not significantly increase arithmetic throughput.

The MP variant, despite reducing the number of multiplications, does not outperform hardware-optimized kernels. This suggests that arithmetic complexity alone is not a sufficient predictor of runtime performance.

3.3 Memory Allocation Behavior

We next isolate memory behavior by comparing execution with and without pooling and planning.

Mode	Allocations	Reuse
No Pool	30	0
Pool	3	27

Table 2: Memory allocation behavior.

Pooling and lifetime planning reduce allocation count by approximately 90%, demonstrating effective reuse of gradient buffers. This confirms that the planner correctly identifies non-overlapping lifetimes and enables safe reuse.

However, this reduction in allocations does not translate into significant runtime improvements. This indicates that allocation overhead is not the dominant cost in these workloads.

It is important to note that pooling reduces allocation overhead but does not reduce the total volume of memory accessed. As a result, its impact on runtime is limited compared to compute optimizations.

3.4 Kernel and Memory Interaction

We evaluate the combined effect of kernel execution and memory reuse during forward and backward passes.

At smaller matrix sizes, memory reuse is highly effective, and allocation overhead is largely eliminated. However, as matrix size increases, runtime becomes dominated by compute cost, and the impact of memory optimization diminishes.

Kernel choice also affects memory behavior. Packed kernels improve locality but may require additional temporary buffers for data reorganization. The MP variant introduces intermediate matrices, increasing memory traffic and reducing cache efficiency.

These observations highlight that compute and memory optimizations are not independent. While memory reuse reduces allocation overhead, compute efficiency is primarily determined by how well operations align with hardware execution characteristics.

3.5 Why Algorithmic Improvements Fail in Practice

Although the MP variant reduces the number of multiplications, it introduces additional additions and temporary buffers. On modern CPUs, performance is often limited by memory bandwidth and cache behavior rather than arithmetic throughput.

As a result, reducing arithmetic operations does not necessarily lead to faster execution. In contrast, packed + SIMD kernels align closely with hardware execution models, maximizing data reuse and vector utilization.

This explains why algorithmic approaches that reduce theoretical complexity may underperform in practice, especially at moderate problem sizes.

3.6 Scheduler Behavior

The scheduler selects kernels based on matrix size using heuristic thresholds. In most cases, it correctly chooses the packed + SIMD kernel for larger workloads.

However, mismatches occur at smaller sizes, where simpler kernels may outperform due to lower overhead. This reflects the limitation of static heuristics and suggests that dynamic profiling could improve selection accuracy.

Overall, the scheduler provides a practical tradeoff between simplicity and performance, while exposing the variability of kernel behavior across input sizes.

4 Limitations

PoolGrad is a simplified runtime designed for controlled experimentation rather than production deployment. Several limitations remain.

First, the kernel implementations, while representative, do not include advanced optimizations such as multi-level cache blocking, register-level microkernels, or architecture-specific tuning found in production libraries like BLAS.

Second, the scheduler relies on heuristic thresholds rather than dynamic profiling or cost modeling. As a result, it may select suboptimal kernels for certain input sizes.

Third, memory optimization is limited to gradient buffers. Activation memory is not reused, and no global memory planning is performed across the entire execution graph.

Finally, the system is restricted to CPU execution and does not consider GPU or heterogeneous environments, where memory and compute tradeoffs may differ significantly.

5 Conclusion

We presented PoolGrad, a minimal runtime for studying the interaction between computation, memory behavior, and execution strategy in neural network training.

Through controlled experiments, we showed that hardware-aligned optimizations such as data packing and SIMD vectorization dominate performance, while memory optimizations primarily reduce allocation overhead without significantly affecting runtime. Algorithmic approaches that reduce multiplication count were found to be highly sensitive to memory and execution overhead, often underperforming compared to hardware-optimized kernels.

These results emphasize that performance in modern systems is not determined solely by arithmetic complexity, but by how effectively computation aligns with hardware characteristics, particularly memory access patterns and vectorization.

PoolGrad provides a simple and reproducible framework for exploring these tradeoffs, offering insight into the relative importance of compute, memory, and scheduling decisions in machine learning systems.